D15A- 04
ML-DL Experiment No. 10

To explore and implement Autoencoders for image denoising using MNIST dataset

1. Dataset Source

Dataset: MNIST Handwritten Digits Dataset
Source: Kaggle
Link: https://www.kaggle.com/datasets/hojjatk/mnist-dataset

2. Dataset Description

The MNIST dataset consists of grayscale images of handwritten digits (0–9), widely used for image processing and deep learning tasks.

Dataset Characteristics

Feature	Description
Total Images	70,000
Training Images	60,000
Testing Images	10,000
Image Size	28 × 28 pixels
Channels	1 (Grayscale)
Classes	10 (digits 0–9)

Input Features

- Pixel intensity values (0–255)

Target Variable

- Not explicitly required for autoencoder
- Model learns to reconstruct input images

Key Characteristics

- Simple and clean dataset
- Suitable for image reconstruction tasks
- Used for unsupervised learning

3. Mathematical Formulation of Autoencoder

1. Encoding Function

$$h = f(Wx + b)$$

2. Decoding Function

$$h = f(Wx + b)$$

3. Reconstruction Loss

$$L = \|x - \hat{x}\|^2$$

4. Algorithm Limitations

- Requires large dataset for better learning
- Output may be blurry for complex images
- Sensitive to noise level
- May overfit if architecture is too complex
- Not suitable for highly detailed reconstruction tasks

5. Methodology / Workflow

Steps Performed

1. Load MNIST dataset in Google Colab

```
[1]
8s
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
```

```
[2]
0s
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

2. Normalize image pixel values

```
[3]
0s
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
```

3. Reshape images into 28×28×1 format

```
[4]
✓ 0s
x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))
```

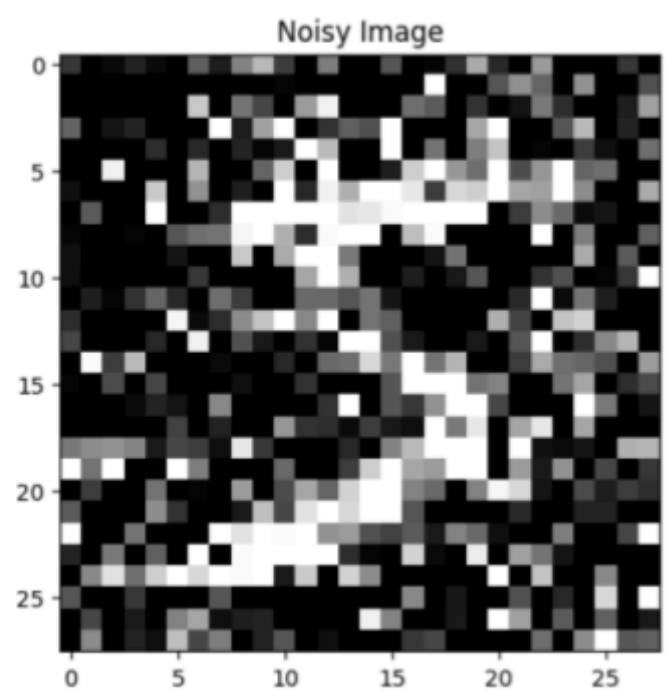
4. Add Gaussian noise to images

```
[5]
✓ 2s
noise_factor = 0.5

x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)

# Clip values
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)
```

```
[6]
✓ 0s
plt.imshow(x_train_noisy[0].reshape(28,28), cmap='gray')
plt.title("Noisy Image")
plt.show()
```



5. Build autoencoder architecture (encoder + decoder)

```
[7]
✓ 0s
model = models.Sequential()

# Encoder
model.add(layers.Conv2D(32, (3,3), activation='relu', padding='same', input_shape=(28,28,1)))
model.add(layers.MaxPooling2D((2,2), padding='same'))

model.add(layers.Conv2D(32, (3,3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D((2,2), padding='same'))

# Decoder
model.add(layers.Conv2D(32, (3,3), activation='relu', padding='same'))
model.add(layers.UpSampling2D((2,2)))

model.add(layers.Conv2D(32, (3,3), activation='relu', padding='same'))
model.add(layers.UpSampling2D((2,2)))

model.add(layers.Conv2D(1, (3,3), activation='sigmoid', padding='same'))

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarnir
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

6. Compile model using optimizer and loss function

```
8]
0s
model.compile(optimizer='adam', loss='binary_crossentropy')
```

7. Train model using noisy images as input and clean images as output

```
[9]
✓ 11m
model.fit(x_train_noisy, x_train,
          epochs=5,
          batch_size=128,
          shuffle=True,
          validation_data=(x_test_noisy, x_test))

... Epoch 1/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 122s 256ms/step - loss: 0.1663 - val_loss: 0.1181
Epoch 2/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 141s 255ms/step - loss: 0.1138 - val_loss: 0.1090
Epoch 3/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 141s 252ms/step - loss: 0.1082 - val_loss: 0.1053
Epoch 4/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 120s 255ms/step - loss: 0.1053 - val_loss: 0.1038
Epoch 5/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 141s 253ms/step - loss: 0.1034 - val_loss: 0.1016
<keras.src.callbacks.history.History at 0x7f531351f800>
```

8. Predict denoised images

```
[10]
✓ 10s
decoded_imgs = model.predict(x_test_noisy)

... 313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 20ms/step
```

9. Visualize noisy vs reconstructed image

10. Predict denoised images

```
[10]
✓ 10s
decoded_imgs = model.predict(x_test_noisy)
313/313 ━━━━━━━━━━━━━━━━━━━━ 7s 20ms/step
```

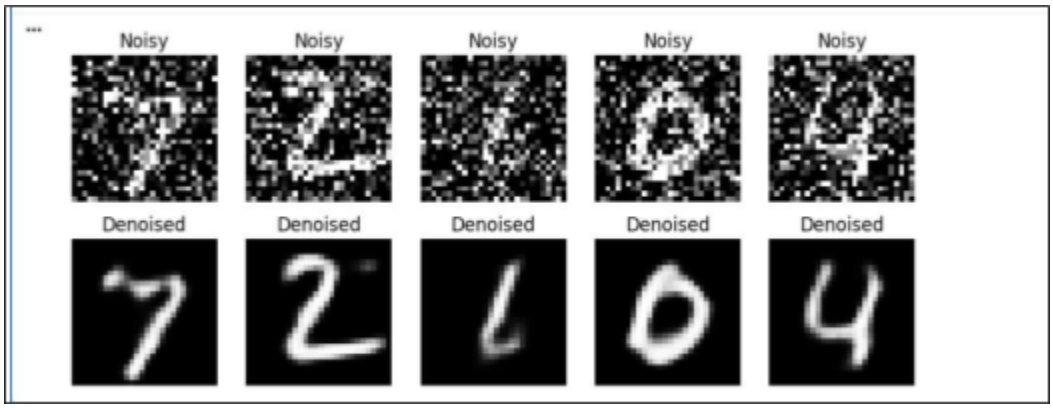
11. Visualize noisy vs reconstructed images

```
[11]
n = 5
plt.figure(figsize=(10,4))

for i in range(n):
    # Noisy
    ax = plt.subplot(2, n, i+1)
    plt.imshow(x_test_noisy[i].reshape(28,28), cmap='gray')
    plt.title("Noisy")
    plt.axis('off')

    # Denoised
    ax = plt.subplot(2, n, i+1+n)
    plt.imshow(decoded_imgs[i].reshape(28,28), cmap='gray')
    plt.title("Denoised")
    plt.axis('off')

plt.show()
```



Workflow Representation

Dataset → Preprocessing → Noise Addition → Autoencoder Model → Training → Reconstruction → Evaluation

6. Performance Analysis

Evaluation Metrics

Metric	Description
Loss	Reconstruction error
Visual Output	Quality of denoised images

Observed Results

- Noisy images are successfully denoised
- Output images retain major features
- Loss decreases during training

Interpretation

- Autoencoder learns to filter noise
- Model captures important patterns in images
- Some blurring occurs due to compression

7. Hyperparameter Tuning

Parameters Tuned

Parameter	Values Tested
Epochs	5, 10
Batch Size	64, 128
Noise Factor	0.3, 0.5
Filters	32, 64

Model Architecture

```
model = Sequential()
# Encoder
Conv2D(32, (3,3), activation='relu', padding='same')
MaxPooling2D((2,2), padding='same')
# Decoder
UpSampling2D((2,2))
Conv2D(1, (3,3), activation='sigmoid', padding='same')
```

Impact of Tuning

- Higher epochs improve reconstruction quality
- More filters improve feature extraction
- Noise factor affects difficulty of task
- Larger batch size speeds training

Best Configuration

Epochs: 5,
Batch Size: 128,
Noise Factor: 0.5,
Filters: 32

Conclusion

The autoencoder model was successfully implemented for image denoising using the MNIST dataset. The model effectively removed noise and reconstructed clean images, demonstrating the capability of autoencoders in unsupervised learning and image processing tasks.